

Comparação entre as LLMs
Critérios de comparação obrigatórios entre as LLMs:

Critério	Descrição
Corretude numérica	Resultado bate com cálculo manual para treliça-teste de 3 barras?
Robustez	Funciona para N = 4, 8, 16, 32 sem alterações?
Qualidade dos gráficos	Legíveis, com legendas, escala adequada?
Eficiência	Tempo de execução para N = 32?
Qualidade do código	Modularização, comentários, tratamento de erros
Aderência ao prompt	Implementou Gauss manualmente ou trapaceou usando solve()?

Tabela Comparativa de LLMs / Códigos

Critério	Código 1	Código 2	Código 3
Corretude Numérica	Alta: Pivoteamento parcial robusto.	Alta: Eliminação progressiva clássica.	Média: Gauss-Jordan sem pivoteamento por valor máximo.
Robustez	Excelente: Usa listas de forma eficiente.	Boa: Estrutura lógica clara para grandes matrizes.	Risco: List comprehension em matrizes 64 x 64 pode ser lenta.
Qualidade Gráfica	Superior: 4 subplots, uso de coolwarm (mapa de calor).	Boa: Gráficos separados, intuitivo.	Básica: Cores binárias (Azul/Vermelho), sem mapa de calor real.
Eficiência	Alta (Gauss é $O(n^{3/3})$).	Alta (Gauss é $O(n^{3/3})$).	Menor (Gauss-Jordan é $O(n^3)$).
Qualidade do Código	Limpo, mas a entrada de dados é repetitiva.	Melhor: Bem documentado, modularizado em funções.	Híbrido: Mistura listas manuais com numpy.
Aderência ao Prompt	Total: Gauss 100% manual.	Total: Gauss 100% manual.	Parcial: Trapaceou usando numpy para estatísticas.

Análise Detalhada por Código

Código 1:

Este código se destaca pela apresentação dos dados.

- Destaque: É o único que utiliza subplots (`plt.subplots(2, 2)`), o que permite uma análise comparativa imediata em uma única janela. O uso de um `Normalize` com `coolwarm` cria um mapa de calor profissional.
- Ponto Fraco: A entrada de dados para cargas é obrigatória para todos os nós, o que torna o teste de $N=32$ extremamente cansativo para o usuário.

Código 2:

Este código é o mais próximo de um software comercial em termos de arquitetura.

- Destaque: Modularização excelente. Ele pergunta quantos nós possuem carga, evitando que o usuário digite 0 0 trinta vezes. Inclui um relatório técnico textual muito completo após os gráficos.
- Pivoteamento: A implementação manual do Gauss é elegante e trata a matriz aumentada de forma clássica.
- Tratamento de Erros: Mais robusto na manipulação de strings de entrada.

Código 3:

Embora funcional, ele falha em seguir a restrição de "não usar bibliotecas de álgebra linear" de forma estrita.

- O Erro: O prompt pediu implementação manual. Embora o solver seja manual, ele utiliza `import numpy as np` para calcular `argmax`, `mean` e magnitudes. Se o ambiente não tiver NumPy instalado, o código quebra, ao contrário dos outros dois que são "Pure Python".
- Visual: O gráfico de "intensidade" apenas aumenta a espessura da linha, o que pode poluir a imagem em treliças complexas ($N=32$).

Veredito Final

Se você precisa de precisão e documentação, use o Código 2.

Se você precisa de visualização para um relatório, use o Código 1.

Nota sobre a Robustez para $N = 32$:

Todos os códigos enfrentarão o mesmo problema: a entrada de dados manual via console.

Para 32 nós, você teria que digitar coordenadas e conectividades centenas de vezes. Para casos reais desse porte, o ideal seria adaptar o código para ler um arquivo `.csv` ou `.json`.

Análise feita com apoio do Gemini.